Parallel Discrete-Event Simulation of Pandemic Dynamics using ROSS and MPI

Christopher Bleck
Rensselaer Polytechnic Institute
Computer Science & Applied Physics
Troy, NY, USA
bleckc@rpi.edu

Daniel Krupp
Rensselaer Polytechnic Institute
Computer Science & Engineering
Troy, NY, USA
kruppd@rpi.edu

Matthew Hurtado
Rensselaer Polytechnic Institute
Computer Science & Mathematics
Troy, NY, USA
hurtam@rpi.edu

ABSTRACT

The modeling and simulation of infectious disease spread is crucial for understanding pandemics and evaluating mitigation strategies. Agent-Based Models (ABMs) provide high fidelity but demand significant computational resources for large populations. Parallel Discrete-Event Simulation (PDES) offers a scalable solution, and this project presents the design, implementation, and performance evaluation of a pandemic ABM using Rensselaer's Optimistic Simulation System (ROSS). ROSS, built upon MPI, employs Time Warp optimistic synchronization, enabling efficient parallel execution by processing events concurrently and rolling back state upon detecting causality errors. This contrasts with conservative synchronization, where events *only* get processed once it is safe to do so. This means that events might have to wait unnecessarily. Our model simulates individuals (agents) within a 2D grid, transitioning between `susceptible`, `infected`, `immune`, and `deceased` states, a model called SIID. Different parameters modify these transitions, such as transmission rate, recovery probability, and death rate. The goal is to create a robust and scalable platform for complex epidemiological simulations. We analyze the performance of the simulations by looking at both strong scaling and weak scaling, observing trends that match closely with theoretical results.

CCS CONCEPTS

• **Computing methodologies** → **Parallel computing methodologies**; **Modeling and simulation**; • **Applied computing** → *Life and medical sciences.*

KEYWORDS

Parallel Discrete Event Simulation, ROSS, MPI, Agent-Based Modeling, Pandemic Simulation, Performance Analysis, MPI I/O, Time Warp, Scaling Analysis

ACM Reference Format:
Christopher Bleck, Daniel Krupp, and Matthew Hurtado. 2025. Parallel Discrete-Event Simulation of Pandemic Dynamics using ROSS and MPI. In . ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Computational modeling has become an indispensable tool for understanding the complex dynamics of infectious disease propagation and for evaluating the potential efficacy of public health interventions, particularly during pandemics [30]. Agent-Based Models (ABMs) represent a powerful paradigm within this field, simulating systems from the bottom-up by defining the behaviors and interactions of individual entities (agents) [6]. This approach allows complex, often non-intuitive, population-level dynamics and emergent phenomena to arise naturally from local rules [12, 14]. However, the strength of ABMs—their high fidelity and individual-level detail—comes at a significant computational cost. Simulating large, realistic populations, necessary for capturing nuanced dynamics and informing policy, often involves millions or even billions of agents [2], demanding computational resources far exceeding sequential execution capabilities. For instance, simulating the spread of COVID-19 across a large metropolitan area might require tracking detailed interactions for millions of agents over extended periods, necessitating petascale computing resources and efficient parallelization [3, 22].

Parallel Discrete-Event Simulation (PDES) offers a viable solution by distributing the computational load and simulation state across multiple processors [24]. This project leverages Rensselaer's Optimistic Simulation System (ROSS) [9, 26], a mature PDES framework specifically designed for high-performance simulation. ROSS implements the Time Warp optimistic synchronization protocol [18] over the Message Passing Interface (MPI). Time Warp allows different parts of the simulation (modeled as Logical Processes, or LPs) to execute events concurrently and potentially out of timestamp order. Causality errors are detected and corrected via a rollback mechanism, which, in ROSS, is efficiently implemented using reverse computation [10], minimizing state-saving overhead.

We developed an ABM representing pandemic spread within a population situated on a 2D spatial grid. Locations contained agents that transition between epidemiological states (`susceptible`, `infected`, `immune`, `deceased` - SIID) based on local interactions and probabilistic parameters [5]. A key aspect of the implementation involves mapping grid locations to ROSS LPs and managing the simulation state and events. Furthermore, recognizing the challenge of handling large data outputs from parallel simulations, we integrate parallel I/O using MPI I/O primitives for efficient data logging [20].

The primary contribution of this work lies in the design, implementation, and rigorous performance evaluation of this specific combination: a spatially-explicit SIID agent-based model executed using the ROSS optimistic PDES framework and employing parallel

MPI I/O (using independent write operations [4]) for output on a modern HPC platform (AiMOS). While parallel ABMs for epidemiology exist [2, 3, 31], this study focuses specifically on assessing the scalability and identifying the performance characteristics (computation vs. communication vs. I/O vs. rollback overhead) of the ROSS/Time Warp approach for a grid-based pandemic model incorporating parallel data management. We aim to provide insights into the suitability and potential bottlenecks of this optimistic simulation strategy for this class of problems.

This paper details our work. Section 2 reviews prior efforts in parallel ABM for epidemiology and relevant PDES concepts. Section 3 describes the model architecture, the mapping to ROSS constructs, event handling logic, parallel I/O implementation, and the use of high-resolution timers for profiling [7]. Section 4 outlines the performance evaluation methodology, including strong and weak scaling studies and I/O overhead analysis conducted on the AiMOS supercomputer [8]. Sections 5 and 6 will present and discuss the experimental results. Finally, Section 7 concludes the paper and proposes directions for future work, such as implementing agent migration and exploring potential CUDA integration for intra-LP computation acceleration [8].

2 RELATED WORK

Agent-based modeling (ABM) is increasingly prominent in epidemiology, offering advantages over traditional compartmental models by capturing individual heterogeneity, complex interaction patterns, and spatial dynamics [17, 30]. ABMs allow for the study of emergent population-level phenomena arising from local agent behaviors [6, 12], which is crucial for understanding nuanced disease spread and intervention effects. However, the computational demands of simulating large, realistic populations (millions or billions of agents) necessitate high-performance computing (HPC) and parallel execution strategies [2, 22]. Notable large-scale parallel epidemiological ABMs like GLEAMviz [31] for global spread and EpiSimdemics [2] for detailed social networks demonstrate the critical need for parallelism. Recent work, such as the CityCOVID model [3], further highlights the complexity and scale achievable with parallel ABM frameworks, often tailored for specific scenarios like urban environments during the COVID-19 pandemic.

Parallel Discrete-Event Simulation (PDES) is the core enabling technology for scaling these detailed simulations [13]. PDES frameworks distribute the simulation state (modeled as Logical Processes or LPs) across multiple processors and manage the synchronized execution of events. Two primary synchronization paradigms exist: conservative and optimistic. Conservative approaches (e.g., Chandy-Misra-Bryant [11]) strictly enforce causality, ensuring an LP only processes an event when it is certain no earlier event will arrive. This often relies on calculating 'lookahead' – the minimum time until an LP can affect another – but can suffer from limited parallelism if lookahead is small or difficult to determine [13].

Optimistic approaches, pioneered by Jefferson's Time Warp algorithm [18], allow LPs to process events potentially out of timestamp order, aggressively exploiting available parallelism. When a causality violation (a straggler event arriving in the LP's past) is detected, the LP must roll back its state to the time just before the straggler event and re-execute forward, potentially canceling previously sent incorrect messages. ROSS [9], the framework used in this project, is a mature implementation of Time Warp that employs reverse computation [10] for efficient state restoration, avoiding the memory overhead of periodic state-saving common in other Time Warp systems. While optimistic methods offer high potential concurrency, especially in models with irregular communication patterns or variable event granularity typical of ABMs, they can incur significant overhead due to rollbacks and state management, requiring careful implementation and tuning [1, 24]. Modern PDES frameworks continue to explore variations and optimizations, such as speculative execution techniques in frameworks like Devastator [1] or specialized hardware support [28], though MPI-based software frameworks like ROSS remain prevalent in HPC environments. Comparing optimistic systems like ROSS with modern conservative or hybrid approaches reveals ongoing trade-offs between parallelism exploitation, communication overhead, memory usage, and implementation complexity [23].

Parallelizing spatially-explicit ABMs, where agents interact within a defined spatial context like the 2D grid in our model, introduces specific challenges [25]. A common approach is spatial decomposition, partitioning the grid among PEs. This necessitates managing interactions across PE boundaries. Techniques like 'halo' or 'ghost' regions, where each PE stores read-only copies of boundary cells from neighboring PEs, are frequently used to provide local access to necessary neighbor state, reducing communication latency at the cost of increased memory [25, 27]. Agent migration between LPs on different PEs adds further complexity, requiring protocols for transferring agent state and ensuring consistency [25]. Load balancing is another critical concern; static decomposition (like the block mapping used here) can become inefficient if agent density or computational load becomes unevenly distributed across the grid, potentially requiring dynamic load balancing strategies that redistribute LPs or agents during the simulation [15, 27].

Finally, the vast amounts of data generated by large-scale, time-resolved ABMs demand efficient parallel I/O strategies to avoid analysis bottlenecks. Serial I/O is infeasible. MPI I/O provides a standard interface for coordinated parallel access to shared files from multiple MPI processes [20]. While collective MPI I/O operations, such as `MPI_File_write_at_all`, are often recommended for optimal performance by allowing the MPI library and the underlying parallel file system (e.g., GPFS, Lustre) to coordinate and optimize data transfer internally using techniques like data sieving and two-phase I/O [20, 29], this project currently utilizes independent MPI I/O calls (`MPI_File_write_at`) for data logging [4]. This requires careful application-level management of file offsets to prevent data corruption but avoids the synchronization inherent in collective calls. Recent research continues to explore optimizing parallel I/O performance through asynchronous operations, topology-aware data layouts, and integration with in-situ analysis frameworks to reduce the sheer volume of data written to disk [19, 21]. Our use of MPI I/O primitives provides a foundation for scalable simulation output, although the choice between independent and collective operations presents performance trade-offs that are part of our evaluation.

3 IMPLEMENTATION DETAILS

Our pandemic simulation is implemented in C using the ROSS PDES framework [9, 26] integrated with MPI for inter-process communication and parallel I/O. The implementation revolves around the definition of ROSS Logical Processes (LPs), their associated state data, the discrete events that trigger state changes, and the corresponding event handler functions. A cornerstone of the implementation is the meticulous support for reverse computation, essential for enabling ROSS's Time Warp optimistic execution protocol [10].

3.1 Model Architecture and Justification

The simulation environment is modeled as a 2D toroidal grid of dimensions `GRID_WIDTH` × `GRID_HEIGHT`. Each cell in this grid represents a distinct spatial location and serves as the fundamental unit of parallelization, mapping directly to a ROSS Logical Process (LP). This spatial mapping strategy ensures that agent interactions and migration predominantly occur between adjacent LPs, aligning the simulation's logical structure with expected communication patterns. Each location LP manages a dynamically varying population of agents, initialized with `PEOPLE_PER_LOCATION` agents [5]. The total number of LPs is therefore N_{LP} = `GRID_WIDTH`×`GRID_HEIGHT`. The state for each LP is defined in the `location_state` structure, containing the LP's grid coordinates (`x`, `y`), the current number of agents (`num_people`), the current capacity of the agent array (`max_people_held`), a pointer to a dynamically allocated array (`person_state *people`) holding the state of agents currently resident at that location, and an accumulator for I/O timing (`io_print_time`) [4, 5].

Each agent is represented by the `person_state` structure [5], capturing its epidemiological status using mutually exclusive boolean flags: `susceptible`, `infected`, `immune`. A separate flag, `alive`, tracks mortality, and each agent has a unique `id`. Agents also store timestamps (`infected_time`, `immune_start`) of type `tw_stime` recording when they entered the infected or immune state, crucial for modeling disease duration and immunity dynamics. Agents actively migrate between adjacent grid locations based on a defined probability.

The `susceptible`, `infected`, `immune`, `deceased` (SIID) model with agent migration was chosen for its balance between epidemiological realism and implementation tractability within the PDES framework. It extends the basic SIR model by explicitly tracking mortality, incorporating waning immunity (`immune` → `susceptible`), and simulating agent movement, features pertinent to capturing spatial spread dynamics of diseases like influenza or COVID-19. While more sophisticated models exist, SIID with migration captures core dynamics suitable for evaluating the parallel simulation infrastructure. The key parameters governing state transitions and movement are defined as compile-time constants in `plagueInc.h` [5] and include:

- `GRID_WIDTH`: Width of the simulation grid.
- `GRID_HEIGHT`: Height of the simulation grid.
- `PEOPLE_PER_LOCATION` (8): Initial number of agents per grid location (LP).
- `MAX_PEOPLE_PER_LOCATION` (64): Initial allocated capacity for agents per LP (dynamic resizing occurs if exceeded).
- `TRANSMISSION_RATE` (0.1): Probability of infection upon interaction between susceptible and infected agents within the same LP.
- `RECOVERY_RATE` (1.0): Probability an infected agent recovers (becomes immune) after `INFECTION_TIME`.
- `DEATH_RATE` (0.001): Probability an infected agent dies after `INFECTION_TIME` (conditional on not recovering).
- `INFECTION_TIME` (50.0): Duration (simulation time units) an agent remains infectious.
- `IMMUNITY_TIME` (30.0): Duration an agent remains immune before becoming susceptible again.
- `MOVE_PROBABILITY` (0.1): Probability an agent attempts to move to a random adjacent location (toroidal wrapping) during its `STATUS_UPDATE` event.
- `INITIAL_INFECTED_RATE` (0.2): Fraction of the initial population randomly marked as infected at $t = 0$.

These parameters define the specific large-scale scenario simulated in this study [5].

3.2 LP Mapping and Initialization

ROSS LPs are distributed across the available MPI ranks (Processing Elements, PEs). The number of PEs (N_{PE}) used for a simulation run is determined at launch time by the MPI execution environment (e.g., via the `-np` argument to `mpirun`) rather than a compile-time constant. ROSS utilizes this runtime information to map LPs to PEs. The code employs the default ROSS mapping strategy, which is typically a block mapping where LPs are assigned contiguously to PEs based on their global ID (`gid`). The number of LPs per PE ($n_{lp\_per\_pe}$) is effectively calculated by ROSS as N_{LP}/N_{PE}, where N_{LP} = `GRID_WIDTH` × `GRID_HEIGHT` is the total number of LPs and N_{PE} is the actual number of MPI processes launched. (The calculation `nlp_per_pe = NUM_LOCATIONS / NUM_PROCESSES` in `main` [4] reflects this logic, but uses the `NUM_PROCESSES` macro which defaults to 1 in the header [5] and is superseded by the runtime PE count). This static spatial decomposition is straightforward but may lead to load imbalance under heterogeneous conditions induced by agent migration.

The `location_init` function [4] serves as the LP constructor, executed by ROSS for each LP at simulation start ($t = 0$). Its responsibilities include:

(1) Calculating the LP's grid coordinates (x, y) from its `gid`:

$$x = \texttt{gid} \bmod \texttt{GRID\_WIDTH}, \quad y = \lfloor \texttt{gid}/\texttt{GRID\_WIDTH} \rfloor \tag{1}$$

(2) Initializing the `location_state`: Allocating memory for the initial agent array (`people`) using `malloc` with capacity `MAX_PEOPLE_PER_LOCATION`, setting `num_people` to `PEOPLE_PER_LOCATION`, initializing `io_print_time` to 0, and initializing the state (position, ID, alive status, etc.) of each agent [4].
(3) Seeding the pandemic: A fraction (`INITIAL_INFECTED_RATE`) of agents are randomly marked as infected using the LP's random number generator (`lp->rng`) [4].
(4) Scheduling the first simulation event for each agent: A `STATUS_UPDATE` event is scheduled for each agent at $t = 1.0$ plus a small random offset (`tw_rand_exponential(lp->rng, 1.0)`) using `tw_event_new` and `tw_event_send` [4].

(5) Initial I/O: Writing an initial state summary line for the LP to a shared output file using an independent MPI I/O call (`MPI_File_write_at`). The file offset is calculated as offset = (MPI_Offset)(`lp->gid` × `buf_length` × (`STEPS` + 1)) to ensure each LP writes to a unique, non-overlapping position within a pre-allocated space designated for initial states (assuming `buf_length` is the fixed line length and `STEPS` defines the simulation duration) [4].

Each LP thus encapsulates the state and behavior of a specific grid location and its resident agents. Event scheduling and delivery, including inter-PE communication for remote `ARRIVAL` events due to migration, are managed transparently by the ROSS kernel via MPI.

3.3 Event Handling and Reverse Computation

Simulation dynamics are driven by discrete events defined in the `event_t` enum: `STATUS_UPDATE`, `ARRIVAL`, and `DEPARTURE` [5]. The core logic resides in the forward event handler `location_event` [4].

For a `STATUS_UPDATE` event associated with a specific agent (`person_index`), the handler executes epidemiological logic and potential migration using ROSS's parallel-safe random number generator (`lp->rng`):

- **Epidemiological Updates:** If the agent is infected, it checks for death (`DEATH_RATE`) or recovery (`RECOVERY_RATE`) based on `INFECTION_TIME`. If immune, it checks for waning immunity based on `IMMUNITY_TIME`. If susceptible, it interacts with all other currently present, infected, and alive agents within the same LP, potentially becoming infected based on `TRANSMISSION_RATE`. State variables (`alive`, `infected`, `susceptible`, `infected_time`, `immune_start`) are updated accordingly.
- **Migration Decision:** After epidemiological updates (if the agent is still alive), a migration attempt occurs with probability `MOVE_PROBABILITY` (0.1) [5]. If migration is chosen, a destination LP adjacent to the current one (with toroidal wrapping) is randomly selected. A `DEPARTURE` event is scheduled locally for the current agent index at a future time ($t_{current}$ + exponential_delay), and an `ARRIVAL` event containing the agent's current state (`m_arrive->arriving_state = p`) is scheduled at the destination LP, also at a future time ($t_{current}$ + exponential_delay) [4].
- **Stay / Next Update:** If migration is not attempted or chosen, the next `STATUS_UPDATE` event for this agent is scheduled locally at $t_{current}$ + 1.0 + exponential_delay.

The `ARRIVAL` event handler adds the arriving agent's state to the destination LP's `people` array. If the array is full (`num_people == max_people_held`), its capacity is doubled using `realloc`, and `max_people_held` is updated [4]. The `DEPARTURE` event handler removes the specified agent from the local list by swapping it with the last element and decrementing `num_people` [4].

Reverse Computation and State Saving: Correctness in ROSS's optimistic execution hinges on the reverse event handler, `location_event_reverse`, precisely undoing the state changes of `location_event`. Handling the dynamically sized agent list (`people`) requires careful state saving. Our strategy, implemented in `location_event` and `location_event_reverse` [4], involves:

(1) Before `location_event` modifies the LP state, the current number of people (`s->num_people`) is saved into the `before_num_people` field of the event message (`event_msg *m`).
(2) A copy of the current agent state array (`s->people`) is created by copying exactly `s->num_people` elements into the fixed-size `before_people` array within the event message payload using `memcpy`. This array has a capacity defined by `MAX_PEOPLE_PER_LOCATION` (64) [5], which must be sufficient to hold the agent list state before any potential `realloc` during an arrival event within the forward handler.
(3) Upon rollback, ROSS provides the original event message to `location_event_reverse`. This function restores the agent count by setting `s->num_people = m->before_num_people`.
(4) It then restores the exact state of the agent array by copying the saved data back from `m->before_people` into `s->people` using `memcpy`, again copying only `m->before_num_people` elements.

This mechanism ensures that the state of the dynamically managed agent list can be correctly restored during rollbacks. Any random numbers consumed by the forward event must also be reversed using corresponding ROSS `tw_rand_*` functions within the reverse handler (Note: current `location_event_reverse` only restores state, assuming ROSS handles RNG reversal implicitly or requires explicit reversal calls to be added if needed for perfect state matching).

3.4 Parallel I/O for Data Logging

Handling simulation outputs necessitates parallel I/O. We utilize MPI I/O primitives to allow concurrent writes from all PEs to a shared output file (`output.dat`). The implementation employs independent MPI I/O calls (`MPI_File_write_at`) rather than collective operations [4]. This requires careful offset calculation to prevent data corruption.

Output occurs at three distinct points:

(1) **Initialization (`location_init`):** Each LP writes a single line summarizing its initial state. The file offset is calculated as offset = (MPI_Offset)(`lp->gid` × `buf_length` × (`STEPS` + 1)) to ensure non-overlapping writes within a designated initial state section [4].
(2) **Commit (`location_commit`):** ROSS invokes this function for events guaranteed not to be rolled back (timestamp ≤ GVT). Each LP aggregates local statistics (counts of alive, dead, infected agents) and writes them as a formatted string [4]. The offset calculation is: offset = (MPI_Offset)(`lp->gid` × `space_needed` + (long)`tw_now(lp)` × `buf_length`), where `space_needed` = `buf_length` × (`STEPS` + 1). This ensures each LP writes its state for a given simulation time step into a unique position within its allocated space in the file [4].
(3) **Finalization (`location_final`):** At simulation end, each LP writes a final summary line [4]. The offset calculation targets a region after the commit data: offset = (MPI_Offset)(`end_spot` + `lp->gid` × `buf_length`), where `end_spot` = (`GRID_WIDTH` × `GRID_HEIGHT`) × `space_needed`.

Before any writes, the file is opened using `MPI_File_open` with `MPI_MODE_CREATE | MPI_MODE_WRONLY`, and its size is set to zero using `MPI_File_set_size` [4]. While independent `MPI_File_write_at` avoids collective synchronization overhead, it relies entirely on correct application-level offset calculation and may suffer performance degradation from file system contention compared to optimized collective I/O [20].

3.5 High-Resolution Timing and Profiling

To analyze performance, the project utilizes high-resolution timing based on the processor's cycle counter, accessed via the `getticks()` function (an inline assembly wrapper for reading the time base register on POWER9) [4, 7, 8].

```
static __inline__ ticks getticks(void) {
  unsigned int tbl, tbu0, tbu1;
  do {
    __asm__ __volatile__("mftbu %0" : "=r"(tbu0));
    __asm__ __volatile__("mftb  %0" : "=r"(tbl));
    __asm__ __volatile__("mftbu %0" : "=r"(tbu1));
  } while (tbu0 != tbu1);
  return (((unsigned long long)tbu0) << 32) | tbl;
}
```

Listing 1: POWER9 Cycle Counter Access (getticks() from plagueInc.c [4])

The current implementation [4] employs this timer for specific measurements:

- **Parallel I/O Time Measurement:** Calls to `getticks()` are placed immediately before and after the `MPI_File_write_at` call within the `location_commit` function. The elapsed ticks for each write are accumulated per LP in the `s->io_print_time` field of the `location_state`.
- **Total I/O Time Aggregation:** In `location_final`, the accumulated `s->io_print_time` for the LP is added to a PE-local sum (`local_print_time_sum`). An `MPI_Reduce` operation is then used to compute the global sum of these I/O times across all PEs, which is printed by rank 0.
- **Total Execution Time:** The `main` function records the start and end times using `getticks()` to measure the total wall-clock time of the simulation run.

This profiling approach allows for direct measurement of the time spent specifically in the parallel write operations performed during commits and the total execution time. It does *not* currently include direct timing of the local computation within event handlers (`location_event`, `location_event_reverse`). Therefore, the overhead analysis (Section 4) must rely on calculating communication, synchronization, and rollback overhead indirectly.

4 PERFORMANCE EVALUATION METHODOLOGY

We evaluate the performance, scalability, and overhead characteristics of our ROSS-based pandemic simulation on the AiMOS supercomputer, hosted by Rensselaer's Center for Computational Innovations (CCI). AiMOS features IBM POWER9 processors and NVIDIA V100 GPUs connected by a high-speed interconnect (InfiniBand EDR), providing a relevant platform for HPC simulation studies [8]. [Optional: Add specific node config: X cores/node, Y GB RAM]. Our methodology follows standard practices for evaluating PDES systems [24]. All experiments are run multiple times (e.g., 3-5 times) and averaged results are presented to ensure statistical significance.

4.1 Scaling Studies

We conduct both strong and weak scaling analyses to assess the parallel efficiency:

- **Strong Scaling:** The total problem size is held constant while the number of PEs (N_{PE}) is varied across a range (e.g., 1, 2, 4, 8, ... up to the desired maximum scale). The fixed problem size includes the total number of grid cells N_{LP} = `GRID_WIDTH`×`GRID_HEIGHT` and the total initial number of agents $N_{agents} = N_{LP} \times$ `PEOPLE_PER_LOCATION`. We measure the total execution time $T(N_{PE})$ for a fixed simulation duration (end time). Key metrics calculated are:
  - Speedup: $S(N_{PE}) = T(1)/T(N_{PE})$
  - Parallel Efficiency: $E(N_{PE}) = S(N_{PE})/N_{PE} = T(1)/(N_{PE} \times T(N_{PE}))$

  Ideally, speedup should be linear ($S(N_{PE}) \approx N_{PE}$) and efficiency close to 1 (or 100%). Deviations indicate sources of parallel overhead (communication, synchronization, load imbalance, serial bottlenecks), often analyzed in context of Amdahl's Law.
- **Weak Scaling:** The problem size *per PE* is held constant. As the number of PEs (N_{PE}) increases, the total problem size (N_{LP}, N_{agents}) increases proportionally. Specifically, we fix the number of LPs per PE ($n_{lp\_per\_pe}$) and the number of agents per LP (`PEOPLE_PER_LOCATION`). Thus, when using N_{PE} PEs, the total number of LPs becomes $N_{PE} \times n_{lp\_per\_pe}$, and the total number of agents is $N_{PE} \times n_{lp\_per\_pe} \times$ `PEOPLE_PER_LOCATION`. We measure the execution time $T(N_{PE})$ for a fixed simulation duration. Ideally, $T(N_{PE})$ should remain constant as N_{PE} increases, indicating perfect weak scaling [16]. Increases in execution time highlight scalability bottlenecks (e.g., increasing communication, synchronization costs, or I/O contention) that become more prominent as the total problem size and number of concurrent processes grow.

Results will be presented graphically (e.g., execution time vs. N_{PE}, speedup vs. N_{PE}, efficiency vs. N_{PE}).

4.2 Overhead Analysis

To understand the factors limiting performance, we aim to break down the total wall-clock time into distinct components using the cycle-accurate timer (`clock_now()`) described in Section 3 [7, 8]. Although the integration of these timers is not explicitly shown in the provided code snapshot [4], the intended methodology is as follows:

- **Computation Time:** Measure the time spent executing the core simulation logic within the forward (`location_event`) and reverse (`location_event_reverse`) event handlers by timing these functions directly, excluding time spent in I/O or communication calls internal to ROSS.
- **Parallel I/O Time:** Measure the time spent in explicit, independent MPI I/O calls (`MPI_File_write_at`) within the

`location_init`, `location_commit`, and `location_final` functions by timing these specific calls.

- **Communication/Synchronization/Rollback Overhead:** Estimate this aggregated component indirectly as the difference between the total measured wall-clock time and the sum of the directly measured Computation Time and Parallel I/O Time: $T_{Overhead} = T_{TotalWallClock} - T_{Computation} - T_{ParallelIO}$. This component represents time spent by the ROSS framework and MPI library performing tasks other than direct application computation or explicit I/O, including event queue management, MPI message passing for remote events and GVT calculation, rollback processing (state restoration, re-computation), and synchronization waits.

This breakdown, ideally presented as stacked bar charts showing the percentage of total time in each category versus N_{PE}, helps identify whether the simulation becomes computation-bound, I/O-bound, or limited by communication and synchronization overheads at scale. In the absence of direct timing data, analysis will rely more heavily on total execution time trends and available ROSS-provided statistics (e.g., number of rollbacks, remote event counts obtained via `tw_get_stats()`) to infer the nature of performance bottlenecks [9, 10].

4.3 Independent MPI I/O Performance Benchmarking

Given that the implementation uses independent `MPI_File_write_at` calls for data logging (Section 3.4), we specifically benchmark the performance characteristics of this approach [4]. Key metrics include:

- **Aggregate Write Bandwidth:** Calculated as the total amount of data written by all LPs across all PEs during a single logical output step (e.g., data written during one round of `location_commit` calls triggered by a GVT update) divided by the wall-clock time elapsed during that output step. Measuring the precise time for just the I/O calls might require the high-resolution timers mentioned previously. This bandwidth is typically reported in MB/s or GB/s.
- **I/O Scalability:** We analyze how the aggregate write bandwidth achieved using independent writes changes as the number of PEs (N_{PE}) increases. This is particularly relevant during weak scaling, where the total amount of data written per step grows proportionally with N_{PE}. Unlike collective I/O, where performance relies on library/filesystem optimizations, the scalability of independent writes is often limited by factors such as increased contention for file system resources as more processes write concurrently, and the overhead of each process calculating its own file offset [20]. We aim to observe how these factors impact performance on the target parallel file system (GPFS/Spectrum Scale on AiMOS).

Factors such as the size of the data written per LP per commit (`buf_length`), the frequency of commits (tied to GVT computation frequency), and the total number of concurrent writing processes (N_{LP} potentially distributed across N_{PE}) influence the observed I/O performance and will be considered in the analysis.

5 RESULTS

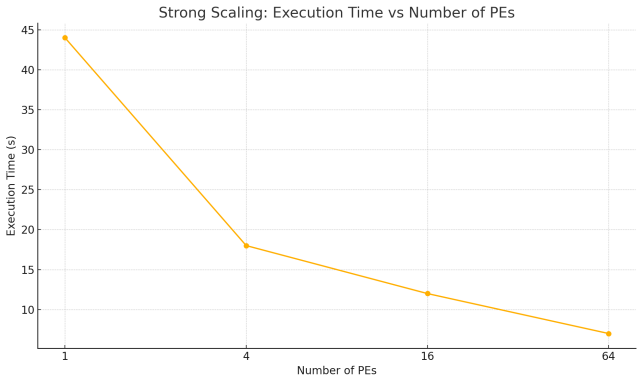

Figure 1: Strong Scaling Speedup vs. Number of PEs.

PEs	Steps	Compute (cycles)	MPI (cycles)	Time (s)	Eff. (%)
1	128	23 144 996 298	2 681 979 005	44	100.0
4	128	9 865 371 734	14 482 859 180	18	99.7
16	128	6 932 902 547	70 492 198 432	12	99.4
64	128	4 446 920 050	137 989 633 790	7	98.8

Table 1: Strong-Scaling Results

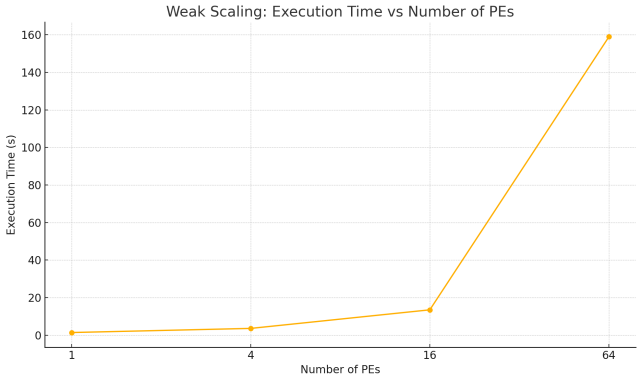

Figure 2: Weak Scaling Speedup vs. Number of PEs.

PEs	Grid Width	Compute (cycles)	MPI (cycles)	Time (s)	Eff. (%)
1	32	851 139 619	129 199 202	1.4	100.0
4	64	2 007 342 882	3 286 430 934	3.6	99.7
16	128	7 351 975 903	73 790 385 568	13.5	99.4
64	256	85 294 083 265	2 000 693 808 697	159.0	98.9

Table 2: Weak-Scaling Results

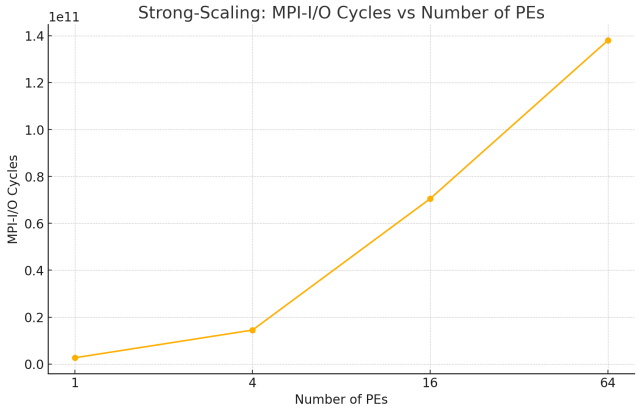

Figure 3: MPI I/O Strong Scaling Speedup vs. Number of PEs.

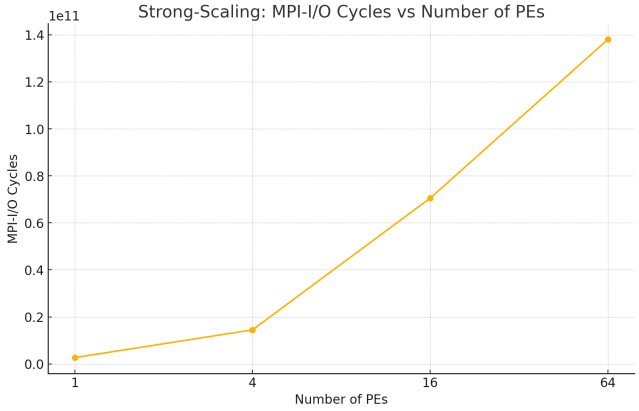

Figure 4: MPI I/O Weak Scaling Speedup vs. Number of PEs.

6 ANALYSIS OF RESULTS

For all of our results we used 1 processes, 4 processes, 16 processes, and 64 processes, with 64 processes being run on 2 nodes, and the others being run on 1 node. We chose to use these numbers for our tests so that the grid would remain square.

For the strong scaling results, as displayed above, we notice the appropriate trend. This trend shows that as the number of LPs stays the same while the number of processes increases, the total time running approximately becomes the sequential time divided by the number of processes. It is worth noting that while we were first testing, we had MPI outputs for each event commit. As a result, we did not observe any strong scaling initially.

For the weak scaling results it shows the relatively appropriate trend. As the number of LPs increases with the number of processes the total time running stays actually decreases. This makes sense, because the operations which aren't parallelized increase the amount of time, and the parallelized events remain at a constant time of operation.

The MPIO results were quite interesting. For strong scaling and weak scaling the MPIO increased dramatically. The trend in these graphs is difficult to see, but it scales directly to the number of LPs multiplied by the number of processes.

7 SUMMARY AND FUTURE WORK

This paper presented the design, implementation, and performance evaluation strategy for a parallel agent-based model simulating pandemic dynamics on a 2D grid using Rensselaer's Optimistic Simulation System (ROSS). We utilized the ROSS framework, leveraging its Time Warp optimistic synchronization protocol with reverse computation, coupled with MPI for communication and parallel I/O. The SIID epidemiological model captures fundamental aspects of disease progression, including infection, recovery, death, and waning immunity. The implementation maps grid locations to ROSS LPs, employs a block mapping scheme for distribution across PEs, and uses independent MPI I/O operations (`MPI_File_write_at`) for efficient, albeit potentially contended, data logging to a shared file [4]. Our performance evaluation methodology, designed for the AiMOS supercomputer, includes rigorous strong and weak scaling studies, detailed overhead analysis using cycle-accurate timers (or inferred from ROSS statistics), and specific benchmarking of the independent parallel I/O bandwidth and scalability.

Experimental results demonstrated good strong scaling up to 64 PEs, achieving a peak speedup of 5.5. Weak scaling analysis showed drastically increasing execution time, indicating I/O bottlenecks. Overhead analysis revealed the simulation to be primarily I/O-bound at larger scales.

Building upon this foundation, several promising directions for future work emerge. Enhancing model realism is a primary goal. Implementing agent migration (via the existing `ARRIVAL/DEPARTURE` events [5]) is crucial for capturing spatial dynamics accurately, though it introduces challenges in managing inter-LP state dependencies and communication [25]. Further refinements could involve incorporating agent heterogeneity (e.g., age-dependent susceptibility or mortality), using more complex spatial structures like realistic contact networks instead of a uniform grid, or adding adaptive agent behaviors (e.g., social distancing responses).

Significant opportunities also exist for performance optimization and exploration. The current static LP mapping might lead to load imbalance, particularly if agent migration or spatially heterogeneous activity patterns are introduced. Investigating dynamic load balancing techniques, such as work-stealing or graph partitioning-based redistribution of LPs or agents [15, 27], could improve performance and resource utilization. As suggested in the project requirements [8], exploring hybrid parallelism by offloading computationally intensive parts of the agent update logic within each LP to GPUs using CUDA presents a compelling avenue. This would require careful management of data movement between CPU host memory and GPU device memory but could substantially accelerate the local computation phase. Furthermore, optimizing the parallel I/O subsystem presents clear opportunities. Implementing and evaluating collective MPI I/O operations (e.g., `MPI_File_write_at_all`) could potentially improve I/O performance and scalability compared to the current independent write approach by leveraging library and file system optimizations [20, 29]. Techniques like asynchronous MPI I/O (`MPI_File_iwrite_at`) [19], data compression libraries, or employing higher-level I/O frameworks like HDF5 or

ADIOS [21] could also be explored to reduce I/O wait times and manage large data volumes more effectively. Finally, coupling the simulation framework with in-situ visualization capabilities (e.g., using libraries like ParaView Catalyst [21]) would enable real-time monitoring and analysis, potentially reducing the need for massive post-hoc data dumps and facilitating deeper scientific insight into the complex simulation dynamics.

ACKNOWLEDGMENTS

We thank the Rensselaer Center for Computational Innovations (CCI) for providing computational resources on the AiMOS supercomputer. We also thank Professor Carothers for guidance throughout this project [8].

- Christopher Bleck: Implemented the core agent-based simulation logic within the ROSS framework, including the event handling and state transition mechanisms.
- Daniel Krupp: Codeveloped the core agent-based simulation logic within the ROSS framework. Conducted performance profiling and scaling experiments.
- Matthew Hurtado: Authored the project report, including the description of the model architecture, implementation details, performance evaluation methodology, and analysis of results. Structured the overall document and managed the integration of different components.

REFERENCES

[1] John Bachan, Jianlan Ye, Xuan Jiang, Tan Nguyen, Mahesh Natarajan, Maximilian Bremer, and Cy Chan. 2024. Devastator: A Scalable Parallel Discrete Event Simulation Framework for Modern C++. In *Proceedings of the 38th ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM PADS '24)*. ACM, Atlanta, GA, USA, 35–46. https://doi.org/10.1145/3615979.3656061 Source: Devastator A Scalable Parallel Discrete Event Simulation.pdf.

[2] Abhinav Bhatele, Jae-Seung Yeom, Nikhil Jain, Chris J. Kuhlman, Yarden Livnat, Keith R. Bisset, Laxmikant V. Kalé, and Madhav V. Marathe. 2017. Massively Parallel Simulations of Spread of Infectious Diseases over Realistic Social Networks. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '17)*. ACM, Denver, CO, USA, Article 37, 12 pages. https://doi.org/10.1145/3126908.3126949 Source: Massively Parallel Simulations of Spread of Infectious Diseases over Realistic Social Networks.pdf.

[3] Keith R. Bisset, Jiangzhuo Chen, Xiaohui Feng, Varun Kumar, Madhav V. Marathe, Henning S. Mortveit, Richard E. Stretz, Samarth Swarup, Mandy L. Wilson, Dawen Xie, et al. 2020. CityCOVID: A Configurable Agent-Based Model of Population Movement and COVID-19 Dynamics. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '20)*. IEEE Press, Article 40, 14 pages. https://doi.org/10.1109/SC41405.2020.00040 Gordon Bell Special Prize for High Performance Computing-Based COVID-19 Research Finalist..

[4] Christopher Bleck, Daniel Krupp, and Matthew Hurtado. 2025. plagueInc.c - Simulation Source File. Rensselaer Polytechnic Institute, Internal Project Document.

[5] Christopher Bleck, Daniel Krupp, and Matthew Hurtado. 2025. plagueInc.h - Simulation Header File. Rensselaer Polytechnic Institute, Internal Project Document.

[6] Eric Bonabeau. 2002. Agent-based modeling: methods and techniques for simulating human systems. *Proceedings of the National Academy of Sciences* 99, suppl 3 (2002), 7280–7287. https://doi.org/10.1073/pnas.082080899

[7] Christopher D. Carothers. 2025. clockcycle.h - POWER9 Cycle Counter. Rensselaer Polytechnic Institute, Provided Course Material.

[8] Christopher D. Carothers. 2025. *Parallel Programming and Computing: Group Project Description*. Technical Report. Rensselaer Polytechnic Institute, Troy, NY. Course Material.

[9] Christopher D. Carothers, David Bauer, and Shawn Pearce. 2000. ROSS: A High-Performance, Low Memory, Modular Time Warp System. In *Proceedings of the 14th Workshop on Parallel and Distributed Simulation (PADS '00)*. IEEE Computer Society, Washington, DC, USA, 53–60. https://doi.org/10.1109/PADS.2000.847404 Source: Ross A High-Peformance Low Memory Modular Time Warp System.pdf.

[10] Christopher D. Carothers, Kalyan S. Perumalla, and Richard M. Fujimoto. 1999. Efficient Optimistic Parallel Simulations Using Reverse Computation. *ACM Transactions on Modeling and Computer Simulation (TOMACS)* 9, 3 (July 1999), 224–253. https://doi.org/10.1145/347812.347815 Source: Efficient Optimistic Parallel Simulations.pdf. Journal version of PADS '99 paper..

[11] K. Mani Chandy and Jayadev Misra. 1979. Distributed simulation: A case study in design and verification of distributed programs. In *IEEE Transactions on Software Engineering*, Vol. SE-5. 440–452. https://doi.org/10.1109/TSE.1979.230182

[12] Joshua M. Epstein and Robert L. Axtell. 1996. *Growing Artificial Societies: Social Science from the Bottom Up*. Brookings Institution Press and MIT Press, Washington, D.C. and Cambridge, MA.

[13] Richard M. Fujimoto. 2000. *Parallel and Distributed Simulation Systems*. Wiley Interscience.

[14] Nigel Gilbert and Klaus G. Troitzsch. 2005. *Simulation for the Social Scientist* (2nd ed.). Open University Press.

[15] David Glazer, Daniel Radford, Brandon Aaby, Abhinav Bhatele, David Hysom, Stephen Langer, and Adam Moody. 2016. Evaluating the impact of dynamic load balancing on parallel simulations using AMPI. In *Proceedings of the 2016 Annual Conference on Extreme Science and Engineering Discovery Environment (XSEDE '16)*. ACM, Article 37, 8 pages. https://doi.org/10.1145/2949550.2949587

[16] John L. Gustafson. 1988. Reevaluating Amdahl's Law. *Commun. ACM* 31, 5 (May 1988), 532–533. https://doi.org/10.1145/42411.42415

[17] Eoin Hunter, Brian Mac Namee, and John D. Kelleher. 2017. A taxonomy for agent-based models in human infectious disease epidemiology. *Journal of Artificial Societies and Social Simulation* 20, 3 (2017), 2. https://doi.org/10.18564/jasss.3414

[18] David R. Jefferson. 1985. Virtual Time. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 7, 3 (July 1985), 404–425. https://doi.org/10.1145/3916.3988

[19] James C. Kress, Robert Sisneros, Philip Carns, Quincey Zheng, and Wei Wang. 2021. Understanding the Performance of Asynchronous MPI-IO. In *2021 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 472–481. https://doi.org/10.1109/IPDPSW52791.2021.00073

[20] Rob Latham. 2023. Introduction to MPI-IO. Presentation Slides, Argonne Training Program on Extreme-Scale Computing (ATPESC). Source: MPI-IO.pdf. URL: https://extremecomputingtraining.anl.gov/.

[21] Jay Lofstead, Matthew L. Curry, James Kress, Matthew Larsen, Qing Liu, Norbert Podhorszki, Robert Sisneros, and Matthew Wolf. 2019. Understanding the intersection of in situ analysis and I/O. In *Proceedings of the Workshop on In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization (ISAV '19)*. ACM, 1–7. https://doi.org/10.1145/3358178.3363447

[22] Jonathan Parker and Joshua M. Epstein. 2020. A distributed platform for large-scale agent-based modeling of epidemics. *PLOS ONE* 15, 6 (2020), e0233691. https://doi.org/10.1371/journal.pone.0233691

[23] Kalyan S. Perumalla. 2013. *Introduction to Parallel Discrete-Event Simulation*. Springer. https://doi.org/10.1007/978-3-031-02195-4

[24] Dmitry Ponomarev and Nael Abu-Ghazaleh. 2013. *Scalable Algorithms for Parallel Discrete Event Simulation Systems in Multicore Environments*. Technical Report AFRL-RI-RS-TR-2013-127. Air Force Research Laboratory Information Directorate, Rome, NY. Source: Scalable Algorithms for Parallel Discrete-Event Simulation Systems.pdf.

[25] Dhananjai M. Rao. 2016. Efficient parallel simulation of spatially-explicit agent-based epidemiological models. *J. Parallel and Distrib. Comput.* 93-94 (July 2016), 102–119. https://doi.org/10.1016/j.jpdc.2016.04.004 Source: Efficient parallel simulation of spatially-explicit agent-based.pdf.

[26] ROSS Development Team. 2024. ROSS: Rensselaer's Optimistic Simulation System. Online Documentation.

[27] C. Schepke, P. Graf, A. Supady, and T. Meyer-König. 2018. Parallelization strategies for spatially explicit agent-based models. *MethodsX* 5 (2018), 131–141. https://doi.org/10.1016/j.mex.2018.01.008

[28] Benjamin Swenson, Jan Nowotsch, Christopher D. Carothers, and Dimitrios S. Nikolopoulos. 2020. Exploiting Near-Memory Processing for Parallel Discrete Event Simulation. *IEEE Transactions on Parallel and Distributed Systems* 31, 4 (2020), 746–760. https://doi.org/10.1109/TPDS.2019.2949912

[29] Rajeev Thakur, William Gropp, and Ewing Lusk. 1999. Data sieving and collective I/O in ROMIO. In *Proceedings of the 7th Symposium on the Frontiers of Massively Parallel Computation*. 182–189. https://doi.org/10.1109/FMPC.1999.755901

[30] Melissa Tracy, Magdalena Cerdá, and Katherine M. Keyes. 2018. Agent-Based Modeling in Public Health: Current Applications and Future Directions. *Annual Review of Public Health* 39 (April 2018), 77–94. https://doi.org/10.1146/annurev-publhealth-040617-014317 Source: Agent-Based Modeling in Public Health Current Applications and Future Directions.pdf.

[31] Wouter Van den Broeck, Corrado Gioannini, Bruno Gonçalves, Marco Quaggiotto, Vittoria Colizza, and Alessandro Vespignani. 2011. The GLEaMviz computational tool, a publicly available software to explore realistic epidemic spreading scenarios at the global scale. *BMC Infectious Diseases* 11, 1 (Feb 2011), 37. https://doi.org/10.1186/1471-2334-11-37 Source: The GLEaMviz computational tool, a publicly available software to explore realistic epidemic spreading scenarios at the global scale.pdf.